



IA appliquée à la sécurité informatique

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction : IA et cybersécurité, panorama

IA appliquée à la sécurité informatique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi l'IA en cybersécurité ?

Les volumes de données à surveiller (journaux applicatifs, journaux systèmes, flux réseau, alertes issues des multiples capteurs de sécurité) ont dépassé depuis longtemps la capacité d'analyse manuelle d'une équipe humaine. Un centre opérationnel de sécurité (SOC) de taille moyenne peut recevoir chaque jour un nombre d'événements bruts qui se compte en dizaines ou centaines de millions, dont l'écrasante majorité est parfaitement légitime. Trier ce volume à la main, ligne par ligne, n'est tout simplement pas réalisable : c'est ce déséquilibre entre volume de données et capacité d'analyse humaine qui motive le recours au machine learning (ML).

Le ML apporte trois bénéfices complémentaires à la détection de sécurité traditionnelle :

- **Automatisation de la détection de schémas (patterns)** : un modèle peut apprendre à reconnaître des combinaisons de caractéristiques associées à un comportement suspect, sans qu'un humain ait à écrire explicitement chaque règle.
- **Passage à l'échelle** : un modèle entraîné peut classer des millions d'événements en quasi temps réel, une fois l'entraînement effectué, à un coût marginal très faible par événement traité.
- **Détection de menaces nouvelles** : contrairement à une approche par signatures, qui ne reconnaît que ce qui a déjà été observé et catalogué, un modèle entraîné à reconnaître un comportement « normal » peut signaler un écart inhabituel même s'il ne correspond à aucune signature connue — un atout précieux face aux attaques dites « zero-day », c'est-à-dire exploitant une vulnérabilité non encore documentée ni corrigée.

Il faut cependant nuancer d'emblée cet optimisme : le ML n'est pas une baguette magique qui remplace la réflexion sur l'architecture de sécurité. Il s'agit d'un outil supplémentaire, qui vient compléter — et non remplacer — les approches établies (pare-feux, règles de détection par signatures, bonnes pratiques de durcissement des systèmes), et dont l'efficacité dépend étroitement de la qualité des données et de la rigueur de sa mise en œuvre, deux thèmes qui reviendront tout au long de ce module.

2. Deux faces d'une même discipline

L'intersection entre IA et cybersécurité ne se limite pas à « utiliser l'IA pour détecter des attaques ». Elle recouvre en réalité trois dimensions distinctes, qu'il est important de séparer clairement dès l'introduction de ce module :

IA défensive. C'est l'usage le plus intuitif : employer des techniques de ML pour renforcer la détection et la réponse. Cela couvre la détection d'intrusions réseau, la détection de malwares, la détection de phishing, la priorisation automatique des alertes remontées à un analyste, et l'automatisation de certaines étapes de la réponse à incident (les plateformes dites SOAR — *Security Orchestration, Automation and Response*). C'est l'axe central de ce module, développé aux chapitres 3, 4 et 5.

IA comme surface d'attaque. Un système d'IA déployé en production n'est pas seulement un outil de défense : c'est aussi, lui-même, un logiciel qui traite des entrées et produit des sorties, et qui hérite à ce titre de nouvelles catégories de vulnérabilités propres au ML. Un attaquant peut chercher à corrompre les données utilisées pour entraîner un modèle (empoisonnement de données), à construire des entrées spécialement conçues pour tromper un modèle déjà entraîné (attaques adversariales), ou encore à reconstituer tout ou partie d'un modèle propriétaire à partir de ses seules réponses (extraction de modèle). Ce volet, souvent négligé dans les présentations grand public de l'IA en sécurité, fait l'objet du chapitre 6.

IA offensive. Les attaquants eux-mêmes se servent désormais de l'IA pour accroître l'efficacité de leurs opérations : génération automatisée de contenus de phishing plus convaincants et plus difficiles à repérer (moins de fautes, ton mieux adapté au contexte), automatisation de certaines phases de reconnaissance, ou encore assistance à la production de code malveillant. Cette dimension, qui modifie l'équilibre du rapport de force entre attaquants et défenseurs, est également abordée au chapitre 6.

Ce module couvre les trois dimensions, avec un accent particulier sur la défense (chapitres 3 à 5) puis sur la sécurité de l'IA elle-même et son usage offensif (chapitre 6). Garder ces trois axes distincts à l'esprit permet d'éviter la confusion fréquente qui consiste à ne penser l'« IA en sécurité » que sous son seul angle défensif.

3. Cas d'usage défensifs courants

Le tableau suivant recense les cas d'usage défensifs les plus répandus dans l'industrie, en précisant à chaque fois le type de tâche de machine learning mobilisée — une classification que nous détaillerons au chapitre 2.

Cas d'usage	Type de tâche ML	Exemple
Détection d'intrusions réseau	classification supervisée / détection d'anomalies	identifier un trafic réseau anormal
Détection de malwares	classification supervisée	classer un fichier binaire comme sain/malveillant
Détection de phishing	classification de texte (NLP)	analyser le contenu d'un e-mail ou d'une URL
Analyse comportementale des utilisateurs (UEBA)	détection d'anomalies	repérer un compte utilisateur au comportement inhabituel
Priorisation d'alertes SOC	classification / scoring	réduire la fatigue d'alerte des analystes

Ces cinq familles de cas d'usage seront reprises et approfondies dans les chapitres suivants : la détection d'intrusions réseau et l'analyse comportementale (UEBA) font l'objet du chapitre 3, la détection de malwares du chapitre 4, et la détection de phishing — comme cas particulier d'analyse de texte — du chapitre 5. La priorisation des alertes SOC, quant à elle, est un fil conducteur transversal : quel que soit le détecteur utilisé en amont, un modèle de scoring permet de hiérarchiser les alertes produites selon leur probabilité de correspondre à une menace réelle, afin de concentrer l'attention limitée des analystes humains sur les cas les plus critiques plutôt que de les noyer sous un flux indifférencié.

4. Limites et illusions à éviter

Introduire un module entier sur l'IA en cybersécurité impose, dès le départ, de désamorcer certaines attentes excessives qui circulent fréquemment autour du sujet.

Le ML n'élimine pas le besoin d'expertise humaine. Un modèle mal conçu, mal entraîné ou mal évalué produit deux types d'erreurs coûteuses : des **faux positifs** (une alerte est levée alors qu'il n'y a pas de menace réelle), qui provoquent à terme une « fatigue d'alerte » chez les analystes — ceux-ci finissent par ignorer ou traiter superficiellement des alertes devenues trop nombreuses — et des **faux négatifs** (une menace réelle n'est pas détectée), qui peuvent avoir des conséquences directement dommageables. Concevoir, entraîner et surtout évaluer correctement un modèle reste un travail d'expertise qui ne s'improvise pas.

Les données d'entraînement conditionnent tout. Un modèle n'apprend que ce que ses données lui montrent. Un modèle entraîné sur le trafic d'un réseau donné, à une période donnée, généralise souvent mal à un contexte différent (un autre réseau, une autre période, de nouveaux usages) : ce phénomène, appelé **dérive conceptuelle** (*concept drift*), impose une supervision et une réévaluation continues des modèles déployés en production, sujet approfondi au chapitre 2.

L'explicabilité est un enjeu de conception, pas un détail technique. Un modèle dit « boîte noire » (typiquement un réseau de neurones profond) peut atteindre une excellente performance brute tout en restant très difficile à auditer : il est malaisé d'expliquer précisément *pourquoi* il a classé un événement donné comme malveillant. Or un analyste de sécurité — et, dans certains contextes réglementaires, un régulateur — a besoin de comprendre les raisons d'une décision pour la valider, la contester ou en tirer des enseignements. Ce compromis entre performance brute et interprétabilité est un critère de choix de modèle à part entière, et non une simple préférence esthétique.

L'IA n'est pas une solution miracle contre un attaquant déterminé. Un attaquant conscient qu'un système de détection s'appuie sur du ML peut chercher spécifiquement à en exploiter les faiblesses : construire des entrées conçues pour franchir la frontière de décision d'un modèle sans changer son comportement malveillant réel, ou encore empoisonner les données utilisées pour le réentraînement. Ce jeu d'échecs entre défenseurs et attaquants, qui n'a pas d'équivalent aussi direct dans les approches par signatures, est développé en détail au chapitre 6.

5. Articulation avec les autres modules du programme

Ce module ne se lit pas de façon isolée : il s'appuie explicitement sur des notions déjà installées dans d'autres parties du programme, et prépare à son tour des compétences mobilisées ailleurs.

Il s'appuie d'abord sur les notions de trafic réseau (protocoles, flux, journalisation) et de vulnérabilités abordées dans le module *Audit organisation et technique* : comprendre ce qu'est une connexion réseau normale, ou ce que recouvre une vulnérabilité exploitable, est un prérequis pour donner un sens concret aux caractéristiques (*features*) utilisées par les modèles présentés dans les chapitres suivants.

Il prépare ensuite une lecture critique des outils de sécurité modernes — EDR (*Endpoint Detection and Response*), XDR (*Extended Detection and Response*), SIEM (*Security Information and Event Management*) — qui intègrent aujourd'hui massivement des composants de machine learning dans leurs moteurs de détection et de priorisation. À l'issue de ce module, l'objectif n'est pas de savoir reconstruire ces outils, mais de comprendre les principes qui sous-tendent leur fonctionnement interne, afin d'en évaluer plus lucidement les promesses, les limites et les compromis de conception.

À retenir

- L'IA en cybersécurité recouvre trois dimensions : défense, sécurité de l'IA elle-même, et usage offensif par les attaquants.
- Le ML complète mais ne remplace pas l'expertise humaine et les approches par signatures/règles.
- La qualité et la représentativité des données d'entraînement sont le facteur déterminant de la performance réelle d'un système de détection basé ML.

Chapitre 2 — Rappels de machine learning

IA appliquée à la sécurité informatique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Apprentissage supervisé, non supervisé, par renforcement

Le machine learning se répartit classiquement en trois grands paradigmes d'apprentissage, qui se distinguent par la nature des données disponibles et par ce que le modèle cherche à optimiser.

Type	Principe	Exemple en sécurité
Supervisé	apprend à partir d'exemples étiquetés (entrée, sortie attendue)	classifier un fichier comme malware/sain à partir d'exemples déjà étiquetés
Non supervisé	découvre une structure dans des données non étiquetées	regrouper des connexions réseau similaires, détecter des anomalies
Par renforcement	un agent apprend par essai-erreur en maximisant une récompense	automatisation de la réponse à incident (usage encore émergent)

L'**apprentissage supervisé** suppose que l'on dispose déjà d'un jeu de données où chaque exemple est associé à une étiquette connue à l'avance — par exemple des milliers de fichiers déjà catalogués comme « malware » ou « sain » par des analystes ou des outils de réputation. Le modèle apprend une fonction qui associe les caractéristiques d'entrée à l'étiquette de sortie, puis on lui soumet de nouveaux exemples jamais vus pour qu'il prédise leur étiquette. C'est le paradigme le plus utilisé en sécurité pour les tâches de classification (détection de malwares, de phishing), précisément parce que ses performances sont généralement supérieures à celles des approches non supervisées, lorsque des données étiquetées de qualité existent.

L'**apprentissage non supervisé** ne dispose d'aucune étiquette : le modèle doit découvrir seul une structure dans les données (des groupes d'observations similaires, ou des observations atypiques). Ce paradigme est particulièrement utile en sécurité précisément parce que les attaques nouvelles ne sont, par définition, jamais étiquetées à l'avance : on ne peut pas entraîner un modèle supervisé à reconnaître une attaque qu'on n'a encore jamais observée.

L'**apprentissage par renforcement** repose sur un agent qui interagit avec un environnement, reçoit une récompense (positive ou négative) en fonction de ses actions, et ajuste sa stratégie pour maximiser la récompense cumulée dans le temps. Son usage en cybersécurité reste aujourd'hui plus expérimental que les deux paradigmes précédents, mais il commence à apparaître dans des scénarios d'automatisation de la réponse à incident, où un agent doit choisir la meilleure action (isoler une machine, bloquer une adresse IP, escalader vers un humain) en fonction du contexte observé.

2. Classification vs régression

Au sein de l'apprentissage supervisé, on distingue deux grandes familles de tâches selon la nature de la sortie prédite :

- **Classification** : prédire une catégorie parmi un ensemble fini de classes possibles (ex. « malveillant » / « bénin » pour une classification binaire, ou plusieurs familles de malwares pour une classification multi-classe).
- **Régression** : prédire une valeur numérique continue (ex. un score de risque compris entre 0 et 100, ou une estimation du temps avant qu'une vulnérabilité ne soit exploitée).

La distinction n'est pas seulement académique : elle détermine le choix de l'algorithme, la fonction de coût utilisée pendant l'entraînement, et surtout les métriques d'évaluation pertinentes (voir section 5). La plupart des cas d'usage en détection de sécurité relèvent de la classification, le plus souvent binaire (attaque/normal) ou multi-classe (type d'attaque parmi plusieurs catégories connues) ; la régression est en revanche plus fréquente pour des tâches de scoring, où l'on cherche à ordonner des éléments par degré de risque plutôt qu'à les ranger dans des catégories strictes.

3. Cycle de vie d'un projet de machine learning

Un projet de ML appliqué à la sécurité ne se résume jamais à « entraîner un modèle » : c'est un cycle complet, dont chaque étape conditionne la qualité du résultat final.

1. **Collecte et préparation des données** : rassembler les données brutes pertinentes (journaux, captures réseau, échantillons de fichiers), puis les nettoyer — corriger ou supprimer les valeurs aberrantes, gérer les valeurs manquantes, normaliser les échelles numériques pour que des variables de magnitudes très différentes (ex. un nombre d'octets en millions et un indicateur binaire) ne biaisent pas l'apprentissage.
2. **Ingénierie des caractéristiques (feature engineering)** : transformer des données brutes, souvent non structurées (paquets réseau, texte libre, séquences d'appels système), en variables numériques exploitables par un modèle. Cette étape est fréquemment celle qui demande le plus d'expertise métier : un ingénieur en sécurité sait quelles propriétés d'un fichier ou d'une connexion sont susceptibles d'être discriminantes.
3. **Séparation des données** : diviser le jeu de données disponible en un ensemble d'**entraînement** (utilisé pour ajuster les paramètres du modèle), un ensemble de **validation** (utilisé pour ajuster les hyperparamètres et comparer plusieurs modèles) et un ensemble de **test** (utilisé une seule fois, à la toute fin, pour estimer la performance réelle du modèle final). Cette séparation stricte est indispensable pour éviter le surapprentissage (*overfitting*, voir section 6) et pour obtenir une estimation honnête de la performance attendue en production.
4. **Entraînement du modèle** : ajuster les paramètres internes du modèle choisi de manière à minimiser l'écart entre ses prédictions et les étiquettes réelles sur l'ensemble d'entraînement.
5. **Évaluation** sur des données jamais vues par le modèle (l'ensemble de test), en utilisant des métriques adaptées au contexte — un point crucial développé à la section 5, car en sécurité une métrique mal choisie peut donner une fausse impression de succès.
6. **Déploiement et supervision continue** : un modèle en production n'est jamais une œuvre définitive. Il doit être réévalué régulièrement, car les données rencontrées en production dérivent souvent progressivement des données d'entraînement d'origine (concept drift) ; un modèle qui n'est plus supervisé se dégrade silencieusement dans le temps.

module. Aucun de ces algorithmes n'est universellement « meilleur » que les autres : le choix dépend de la nature des données, du volume disponible, du besoin d'interprétabilité et des contraintes de latence en production.

Algorithme	Type	Caractéristique
Régression logistique	supervisé, classification	simple, interprétable, bonne base de référence
Arbre de décision / Forêt aléatoire (Random Forest)	supervisé, classification/régression	robuste, gère bien les données hétérogènes, relativement interprétable
SVM (Support Vector Machine)	supervisé, classification	efficace en haute dimension
K-means	non supervisé, clustering	regroupe des observations similaires
Isolation Forest	non supervisé, détection d'anomalies	isole les observations atypiques efficacement, utilisé en TP2
Réseaux de neurones / Deep Learning	supervisé ou non supervisé	performant sur données complexes (texte, image, séquences) mais moins interprétable

La **régression logistique** reste, malgré sa simplicité, un choix pertinent comme point de départ de tout projet : rapide à entraîner, ses coefficients s'interprètent directement (l'influence de chaque caractéristique sur la prédiction est explicite), ce qui en fait une excellente ligne de base à laquelle comparer des modèles plus complexes.

Les **forêts aléatoires** (*Random Forest*) combinent un grand nombre d'arbres de décision entraînés sur des sous-échantillons légèrement différents des données, puis agrègent leurs votes. Cette approche d'ensemble les rend généralement robustes au bruit et aux données hétérogènes (mélange de variables numériques et catégorielles), tout en conservant une interprétabilité raisonnable via l'importance des caractéristiques.

Les **SVM** cherchent la frontière de séparation qui maximise la marge entre les classes, ce qui les rend particulièrement efficaces lorsque le nombre de caractéristiques (dimensions) est élevé par rapport au nombre d'exemples disponibles — une situation fréquente en analyse de texte ou de binaires.

K-means et **Isolation Forest** sont deux algorithmes non supervisés aux usages différents mais complémentaires en sécurité : K-means regroupe des observations en un nombre fixé de groupes homogènes (utile pour identifier des profils de trafic typiques), tandis qu'Isolation Forest cherche spécifiquement les observations qui se démarquent du reste — une approche naturellement adaptée à la détection d'anomalies, reprise en pratique au TP2 de ce module.

Les **réseaux de neurones**, et plus largement le *deep learning*, excellent sur des données à structure complexe (texte, image, séquences temporelles) où l'ingénierie manuelle des caractéristiques serait difficile ou insuffisante, le modèle apprenant lui-même des représentations

5. Métriques d'évaluation, essentielles en sécurité

Pour une classification binaire, la **matrice de confusion** est l'outil de base pour analyser les erreurs d'un modèle : elle distingue quatre cas possibles selon la valeur réelle et la valeur prédite.

	Prédit positif	Prédit négatif
Réel positif	Vrai Positif (VP)	Faux Négatif (FN)
Réel négatif	Faux Positif (FP)	Vrai Négatif (VN)

À partir de ces quatre quantités, on construit les métriques réellement utilisées pour évaluer un modèle de détection :

- **Précision** = $VP / (VP + FP)$: parmi toutes les alertes levées par le modèle, quelle proportion correspond réellement à une menace ? Une précision faible signifie beaucoup de fausses alertes.
- **Rappel (recall)** = $VP / (VP + FN)$: parmi toutes les menaces réellement présentes, quelle proportion a été détectée par le modèle ? Un rappel faible signifie que des menaces réelles passent inaperçues.
- **F1-score** : moyenne harmonique de la précision et du rappel, utile pour résumer les deux en une seule valeur lorsqu'on souhaite comparer plusieurs modèles sans privilégier a priori l'un des deux critères.
- **Taux de faux positifs** = $FP / (FP + VN)$: la proportion d'éléments réellement bénins qui déclenchent, à tort, une alerte. Cette métrique est particulièrement critique en sécurité, car un taux élevé de faux positifs entraîne rapidement la « fatigue d'alerte » des analystes, qui finissent par traiter les alertes de façon superficielle, voire par les ignorer complètement — ce qui, paradoxalement, dégrade la sécurité globale plutôt que de l'améliorer.

En sécurité, la simple exactitude globale (accuracy) — c'est-à-dire la proportion totale de prédictions correctes — est souvent trompeuse. Il faut ici garder à l'esprit un exemple illustratif classique : sur un jeu de données très déséquilibré, où par exemple seule une infime fraction du trafic est réellement malveillante, un modèle qui prédirait systématiquement « bénin », sans jamais rien détecter, obtiendrait malgré tout une exactitude globale très proche de 100 %, tout en étant totalement inutile puisqu'il ne détecte aucune attaque. C'est précisément parce que les classes sont déséquilibrées en sécurité (les attaques sont rares comparées au trafic légitime) que le rappel et la précision, calculés en particulier sur la classe minoritaire (l'attaque, la classe positive), sont les métriques réellement informatives — et non l'exactitude globale, qui peut masquer un modèle totalement défaillant.

6. Surapprentissage et sous-apprentissage

Deux écueils opposés guettent l'entraînement d'un modèle, et il est essentiel de savoir les diagnostiquer avant tout déploiement :

- **Surapprentissage (overfitting)** : le modèle « apprend par cœur » les particularités — y compris le bruit — des données d'entraînement, au point de perdre sa capacité à généraliser à de nouvelles données. Un modèle en surapprentissage affiche une excellente performance sur les données d'entraînement mais une performance nettement plus faible sur des données jamais vues, symptôme caractéristique d'un modèle trop complexe pour la quantité de données disponible, ou entraîné trop longtemps.
- **Sous-apprentissage (underfitting)** : à l'inverse, le modèle est trop simple pour capturer la structure réelle des données, et ses performances restent médiocres même sur les données d'entraînement elles-mêmes — signe qu'il faut soit enrichir les caractéristiques utilisées, soit choisir un modèle plus expressif.

La **validation croisée** (*cross-validation*), qui consiste à entraîner et évaluer le modèle sur plusieurs découpages différents des données d'entraînement, et un **ensemble de test totalement indépendant**, jamais utilisé pendant l'entraînement ni le réglage des hyperparamètres, permettent de détecter ces deux situations de façon fiable avant tout déploiement en production. Un modèle qui n'a jamais été évalué sur des données réellement indépendantes ne devrait jamais être déployé tel quel dans un système de sécurité opérationnel.

À retenir

- Le choix entre supervisé/non supervisé dépend de la disponibilité de données étiquetées, souvent rare en sécurité pour les attaques nouvelles.
- Précision, rappel et F1-score, pas la seule exactitude, sont les métriques pertinentes pour évaluer un modèle de détection en contexte déséquilibré.
- Le cycle de vie complet (données → features → entraînement → évaluation → supervision continue) doit être maîtrisé, pas seulement l'entraînement du modèle.

Chapitre 3 — Détection d'anomalies et d'intrusions par ML

IA appliquée à la sécurité informatique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Détection par signatures vs détection par anomalies

Un système de détection d'intrusions (IDS) ou de prévention d'intrusions (IPS) peut reposer sur deux philosophies fondamentalement différentes, résumées dans le tableau ci-dessous.

Approche	Principe	Avantage	Limite
Signatures	comparaison à des motifs d'attaques connues	très faible taux de faux positifs sur les menaces connues	ne détecte pas les attaques inédites
Anomalies (ML)	apprentissage du comportement « normal », alerte sur tout écart significatif	peut détecter des menaces nouvelles (zero-day)	taux de faux positifs souvent plus élevé, nécessite une phase d'apprentissage du « normal »

La **détection par signatures** fonctionne comme un antivirus classique : elle compare le trafic ou les événements observés à une base de motifs déjà connus et documentés comme malveillants (une séquence d'octets caractéristique, un motif de requête HTTP correspondant à une vulnérabilité connue, etc.). Tant que le motif recherché correspond exactement à une attaque déjà cataloguée, cette approche est très fiable et génère peu de fausses alertes. Sa faiblesse structurelle est symétrique : elle ne peut, par construction, détecter que ce qui a déjà été observé et ajouté à la base de signatures — une attaque totalement inédite passe inaperçue tant qu'aucune signature ne la couvre.

La **détection par anomalies**, à l'inverse, ne cherche pas à reconnaître des attaques connues mais à modéliser ce à quoi ressemble un comportement « normal » (du trafic réseau habituel, une activité utilisateur typique), puis à signaler tout écart significatif par rapport à ce modèle. Cette approche peut en principe détecter des menaces jamais vues auparavant, y compris des attaques zero-day, précisément parce qu'elle ne dépend pas d'un catalogue préexistant. En contrepartie, elle génère généralement davantage de faux positifs, car tout ce qui est simplement inhabituel — sans être nécessairement malveillant — peut déclencher une alerte, et elle nécessite une phase d'apprentissage préalable du comportement normal, phase elle-même sensible : si des données d'entraînement contiennent déjà des comportements anormaux non détectés, le modèle risque de les considérer, à tort, comme faisant partie de la normalité.

En pratique, les IDS/IPS modernes ne choisissent pas entre les deux approches : ils les **combinent**, en utilisant les signatures comme première ligne de défense rapide et fiable contre les menaces connues, et la détection par anomalies comme filet de sécurité complémentaire pour repérer ce que les signatures ne couvrent pas.

2. Sources de données pour un IDS basé ML

Un système de détection basé ML a besoin de données représentatives de l'activité à surveiller. Trois grandes familles de sources sont couramment mobilisées, souvent en complément les unes des autres :

- **Trafic réseau** : plutôt que d'analyser chaque paquet individuellement, on travaille généralement sur des caractéristiques dérivées des flux réseau (protocole standard NetFlow ou équivalent) : durée de la connexion, protocole utilisé, nombre d'octets échangés dans chaque direction, nombre de paquets, taux d'erreur observé. Cette agrégation par flux réduit considérablement le volume de données tout en conservant l'information pertinente pour la détection.
- **Journaux systèmes/applicatifs** : les journaux produits par les systèmes d'exploitation et les applications (tentatives de connexion, commandes exécutées, accès aux fichiers) fournissent une autre source riche, notamment sous forme de séquences d'événements et de fréquences (par exemple, la fréquence de connexions échouées sur un compte donné peut signaler une tentative de force brute).
- **Comportement utilisateur (UEBA)** : l'analyse du comportement des utilisateurs et des entités (*User and Entity Behavior Analytics*) s'appuie sur des indicateurs comme les horaires de connexion habituels d'un utilisateur, les ressources qu'il accède habituellement, ou le volume de données qu'il télécharge en temps normal — un écart marqué par rapport à ce profil habituel (connexion à 3h du matin, accès à des ressources jamais consultées auparavant, volume de téléchargement anormalement élevé) peut signaler un compte compromis ou une menace interne.

3. Ingénierie des caractéristiques pour le trafic réseau

L'ingénierie des caractéristiques (*feature engineering*), déjà présentée de façon générale au chapitre 2, prend une forme concrète et bien documentée dans le domaine de la détection d'intrusions réseau. Le jeu de données **NSL-KDD**, réutilisé en TP1 de ce module, illustre bien les caractéristiques dérivées classiquement utilisées :

- durée de la connexion ;
- type de protocole (TCP, UDP, ICMP) ;
- service ciblé (HTTP, FTP, SSH...) ;
- nombre d'octets source → destination et destination → source ;
- nombre de tentatives de connexion échouées récentes vers le même hôte ;
- taux de connexions vers un même service dans une fenêtre temporelle glissante.

Le passage de paquets réseau bruts à ce type de caractéristiques agrégées est une étape déterminante, et souvent sous-estimée : **un modèle ne peut jamais être meilleur que les caractéristiques qui lui sont fournies**. Un modèle sophistiqué entraîné sur des caractéristiques peu discriminantes obtiendra de moins bons résultats qu'un modèle simple entraîné sur des caractéristiques pertinentes bien choisies. C'est pourquoi, dans un projet réel, une part importante du temps est consacrée non pas au choix de l'algorithme, mais à la définition et à la validation de ces caractéristiques dérivées, en collaboration étroite avec des experts du domaine réseau.

4. Approches non supervisées de détection d'anomalies

Lorsque des étiquettes fiables (trafic déjà catalogué « normal » ou « attaque ») ne sont pas disponibles — ce qui est fréquent pour les menaces nouvelles — on se tourne vers des approches non supervisées, qui ne nécessitent pas d'étiquettes préalables.

- **Clustering (K-means, DBSCAN)** : ces algorithmes regroupent les observations similaires en clusters. L'hypothèse sous-jacente est que le comportement normal se regroupe naturellement en un ou plusieurs clusters denses, tandis que les comportements anormaux, plus rares et plus hétérogènes, se retrouvent isolés ou dans de petits clusters peu peuplés — ces points isolés sont alors considérés comme suspects.
- **Isolation Forest** : cet algorithme construit un grand nombre d'arbres de décision aléatoires, en sélectionnant à chaque nœud une caractéristique et un seuil de séparation au hasard. L'intuition centrale est qu'une observation atypique, qui se distingue nettement du reste des données sur au moins une caractéristique, nécessite en moyenne beaucoup moins de séparations aléatoires pour être isolée du reste des points qu'une observation typique, noyée dans une région dense. Le nombre moyen de séparations nécessaires devient ainsi directement un score d'anomalie, sans qu'il soit besoin de définir explicitement ce qu'est une observation « normale ».
- **Autoencodeurs (réseaux de neurones)** : un autoencodeur est un réseau de neurones entraîné à reconstruire ses propres entrées après les avoir compressées dans une représentation de dimension réduite. Entraîné uniquement sur du trafic normal, il apprend à bien reconstruire ce type de données ; face à une observation qui s'écarte fortement des motifs appris, l'erreur de reconstruction — l'écart entre l'entrée et sa reconstruction — devient élevée, ce qui constitue un signal d'anomalie potentielle.

5. Approches supervisées

Lorsque des données étiquetées de qualité sont disponibles — c'est-à-dire du trafic déjà catalogué comme normal ou correspondant à des types d'attaques connus, à l'image des jeux de données de référence **NSL-KDD** ou **CICIDS** — une classification supervisée (Random Forest, SVM, réseaux de neurones) obtient généralement de meilleures performances qu'une approche purement non supervisée sur les catégories d'attaques représentées dans les données. Cette meilleure performance a cependant une contrepartie structurelle : un modèle supervisé reste, par construction, limité aux catégories d'attaques présentes dans son jeu d'entraînement, et ne pourra pas, en principe, reconnaître une famille d'attaque radicalement nouvelle qu'il n'a jamais rencontrée — un rôle qui reste alors dévolu aux approches non supervisées ou à une combinaison des deux, souvent utilisée en pratique.

6. Défis pratiques du déploiement en production

Passer d'un modèle qui fonctionne bien sur un jeu de données de test à un système opérationnel en production soulève plusieurs défis pratiques, souvent sous-estimés lors de la conception initiale.

- **Déséquilibre des classes** : les attaques représentent une infime fraction du trafic total observé en conditions réelles, un déséquilibre qui peut biaiser fortement l'apprentissage d'un modèle si rien n'est fait pour le corriger. Des techniques de rééquilibrage — sur-échantillonnage de la classe minoritaire, sous-échantillonnage de la classe majoritaire, ou génération synthétique d'exemples par la méthode SMOTE (*Synthetic Minority Over-sampling Technique*) — ainsi que le choix de métriques adaptées (précision, rappel, F1-score sur la classe minoritaire, comme vu au chapitre 2) sont nécessaires pour obtenir un modèle réellement utile.
- **Dérive conceptuelle (concept drift)** : le trafic considéré comme « normal » évolue continuellement dans le temps — nouveaux usages, nouvelles applications déployées, nouveaux services ouverts — ce qui signifie qu'un modèle figé, entraîné une fois pour toutes, se dégrade progressivement. Un modèle en production doit donc être réentraîné périodiquement sur des données récentes pour rester pertinent.
- **Volumétrie et latence** : un IDS doit souvent traiter des flux en temps réel ou quasi réel, ce qui contraint fortement le choix et la complexité du modèle utilisé : un modèle très sophistiqué mais lent à l'inférence peut être inutilisable si le volume de trafic à analyser dépasse sa capacité de traitement, créant un goulot d'étranglement critique en production.
- **Intégration avec le SOC** : une alerte générée automatiquement n'a de valeur que si elle est réellement exploitable par un analyste humain — elle doit être accompagnée d'un contexte suffisant (quelle caractéristique a déclenché l'alerte, quel est le niveau de confiance du modèle), idéalement d'une explication compréhensible, et intégrée dans un processus de priorisation qui évite de noyer les analystes sous un flux d'alertes brutes et indifférenciées.

À retenir

- Les systèmes de détection combinent en général signatures (précis mais limités aux menaces connues) et ML par anomalies (couvre potentiellement les menaces nouvelles, au prix de plus de faux positifs).
- La qualité des caractéristiques extraites du trafic brut conditionne fortement la performance du modèle.
- Le déséquilibre des classes et la dérive conceptuelle sont deux défis pratiques majeurs, souvent sous-estimés lors du passage du prototype à la production.

Chapitre 4 — Détection de malwares par machine learning

IA appliquée à la sécurité informatique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Limites de la détection par signatures pour les malwares

Les antivirus traditionnels reposent largement sur des **signatures** : des empreintes numériques (typiquement des condensats cryptographiques, ou *hashes*) calculées à partir de fichiers malveillants déjà identifiés et catalogués. Cette approche est efficace tant que le fichier analysé correspond exactement, octet pour octet, à un échantillon déjà répertorié dans la base de signatures.

Cette approche est cependant mise en échec par deux techniques utilisées de longue date par les auteurs de malwares : le **polymorphisme**, qui consiste à modifier automatiquement le code du malware à chaque propagation (par exemple en le chiffrant avec une clé différente à chaque copie) tout en conservant strictement le même comportement malveillant une fois exécuté, et le **métamorphisme**, une variante plus poussée où le code lui-même est réécrit sous une forme différente (réorganisation des instructions, insertion de code mort) sans passer par le chiffrement. Dans les deux cas, la signature calculée sur le fichier change à chaque variante, alors que le comportement réel reste identique — ce qui rend une base de signatures fondée sur l'empreinte exacte du fichier structurellement incapable de suivre le rythme.

À cela s'ajoute un problème de volume : le nombre de nouvelles variantes de malwares produites quotidiennement dans le monde est considérable, ce qui rend la maintenance exhaustive d'une base de signatures à jour pratiquement impossible pour un éditeur de sécurité, quels que soient les moyens engagés. C'est précisément cette limite structurelle qui motive le recours à des approches de détection fondées sur le comportement et les caractéristiques structurelles d'un fichier plutôt que sur son empreinte exacte — l'objet de ce chapitre.

2. Deux approches d'analyse pour l'extraction de caractéristiques

Pour entraîner un modèle de détection de malwares, il faut d'abord extraire des caractéristiques exploitables de chaque échantillon de fichier. Deux grandes familles de méthodes existent, avec des compromis opposés.

Approche	Principe	Avantage	Limite
Analyse statique	examen du fichier sans l'exécuter (structure du binaire, chaînes de caractères, imports de fonctions système, entropie)	rapide, sans risque d'exécution	contournable par obfuscation, chiffrement du code (packers)
Analyse dynamique	exécution du fichier dans un environnement contrôlé (sandbox) et observation du comportement (appels système, connexions réseau, modifications de fichiers/registre)	détecte le comportement réel, résiste mieux à l'obfuscation	plus lente, coûteuse en ressources, certains malwares détectent la sandbox et modifient leur comportement

L'**analyse statique** examine le fichier « à froid », sans jamais l'exécuter : elle inspecte sa structure interne, les chaînes de caractères qu'il contient, les fonctions système qu'il déclare vouloir utiliser (ses imports), ou encore son entropie globale. Elle a l'avantage d'être rapide et de ne présenter aucun risque, puisque le code n'est jamais réellement lancé. Sa faiblesse est qu'un attaquant peut délibérément brouiller la structure du fichier — par obfuscation ou en le chiffrant avec un « packer » (un programme qui compresse et déchiffre le code malveillant uniquement au moment de l'exécution) — pour masquer précisément les caractéristiques qu'une analyse statique chercherait à exploiter.

L'**analyse dynamique**, à l'inverse, consiste à exécuter réellement le fichier suspect dans un environnement contrôlé et isolé (une *sandbox*), afin d'observer son comportement effectif : quels appels système il effectue, quelles connexions réseau il initie, quelles modifications il apporte au système de fichiers ou à la base de registre. Cette approche a l'avantage de capturer le comportement réel du programme, ce qui la rend plus résistante à l'obfuscation du code (un code obfusqué produit, une fois exécuté, le même comportement malveillant qu'un code clair). Elle est en revanche plus lente et plus coûteuse en ressources qu'une simple inspection statique, et certains malwares sont eux-mêmes conçus pour détecter qu'ils s'exécutent dans un environnement d'analyse (sandbox) et modifient alors leur comportement — voire restent totalement inactifs — pour échapper à la détection.

En pratique, la plupart des solutions robustes combinent les deux approches, l'analyse statique servant de premier filtre rapide et l'analyse dynamique intervenant pour les cas ambigus ou lorsque l'obfuscation empêche une analyse statique concluante.

3. Caractéristiques exploitées en analyse statique

Sans exécuter le fichier, plusieurs types de caractéristiques peuvent être extraits et fournis à un modèle de classification :

- **Métadonnées du fichier** : taille du fichier, contenu des en-têtes, structure de la section PE (*Portable Executable*) pour les exécutables Windows — ces en-têtes décrivent notamment les sections du binaire et la façon dont il doit être chargé en mémoire.
- **Entropie du fichier ou de ses sections** : l'entropie mesure le degré de désordre apparent des octets. Une entropie élevée sur une section du fichier peut indiquer la présence de contenu chiffré ou compressé, fréquent dans les malwares « packés » précisément pour échapper à l'analyse statique — une entropie anormalement élevée est donc en elle-même un indicateur exploitable.
- **N-grammes d'octets ou d'instructions assembleur** : des séquences de n octets ou d'instructions consécutives, qui capturent des motifs structurels récurrents dans le code, y compris des motifs propres à certaines familles ou techniques malveillantes.
- **Fonctions de l'API système importées** : la liste des fonctions système qu'un binaire déclare vouloir appeler (par exemple des fonctions liées aux communications réseau, à la manipulation directe de la mémoire, ou à des techniques d'injection dans d'autres processus) fournit souvent un indice fort quant à l'intention du programme, même avant toute exécution.

4. Caractéristiques exploitées en analyse dynamique (sandbox)

Une fois le fichier exécuté dans un environnement d'analyse contrôlé, on observe et on enregistre son comportement effectif, dont plusieurs dimensions peuvent être transformées en caractéristiques pour un modèle :

- **Séquence d'appels système (syscalls) effectués** : l'ordre et la nature des interactions du programme avec le système d'exploitation (ouverture de fichiers, création de processus, accès mémoire) forment une empreinte comportementale souvent plus difficile à masquer que la structure statique du fichier.
- **Connexions réseau initiées** : les domaines contactés et les protocoles utilisés peuvent révéler une communication avec une infrastructure de commande et de contrôle (C2), typique de nombreuses familles de malwares.
- **Modifications du système de fichiers et de la base de registre** : la création, la suppression ou la modification de fichiers et de clés de registre pendant l'exécution documente l'impact concret du programme sur le système.
- **Techniques de persistance observées** : la création de tâches planifiées ou de clés de démarrage automatique au sein de la base de registre trahit souvent une volonté de rester actif après un redémarrage, un comportement caractéristique de nombreux malwares.

5. Modèles utilisés

Une fois les caractéristiques extraites — qu'elles proviennent de l'analyse statique, de l'analyse dynamique, ou d'une combinaison des deux — plusieurs familles de modèles peuvent être utilisées pour la classification :

- **Classification supervisée** (Random Forest, gradient boosting, réseaux de neurones) sur des caractéristiques extraites statiquement et/ou dynamiquement : c'est l'approche la plus répandue dans les solutions commerciales, notamment dans les EDR de nouvelle génération, en raison de son bon compromis entre performance et coût de mise en œuvre.
- **Modèles de séquence** (réseaux de neurones récurrents RNN, ou architectures Transformers) : plutôt que de résumer le comportement observé en sandbox par des statistiques agrégées, ces modèles traitent directement la séquence ordonnée d'appels système, ce qui leur permet de capturer des dépendances temporelles fines (par exemple, un enchaînement précis d'actions caractéristique d'une technique d'attaque particulière) que des caractéristiques agrégées perdraient.
- **Clustering** de familles de malwares : au-delà de la simple classification binaire malveillant/sain, le regroupement d'échantillons similaires (par les caractéristiques extraites) permet d'identifier des familles de malwares apparentées, une information précieuse pour prioriser l'analyse humaine — un nouvel échantillon proche d'une famille déjà bien comprise peut être traité plus rapidement qu'un échantillon isolé, potentiellement inédit.

6. Techniques d'évasion contre les détecteurs ML

Tout comme pour les signatures classiques, les auteurs de malwares adaptent activement leurs techniques pour échapper spécifiquement aux détecteurs fondés sur le ML :

- **Obfuscation et packing** : ces techniques rendent l'analyse statique moins efficace en masquant délibérément les caractéristiques structurelles qu'un modèle chercherait à exploiter, forçant en pratique le recours à l'analyse dynamique.
- **Détection d'environnement virtualisé/sandbox** : de nombreux malwares intègrent des routines qui vérifient s'ils s'exécutent dans un environnement d'analyse automatisé (indices matériels ou logiciels typiques d'une machine virtuelle) et, si c'est le cas, modifient leur comportement ou restent totalement inactifs, rendant l'analyse dynamique inopérante pour ces échantillons.
- **Exemples adversariaux** : une technique plus récente et spécifiquement dirigée contre les modèles de ML consiste à modifier de façon minime un binaire — par exemple en y ajoutant des octets inoffensifs ou en réorganisant certaines sections sans en changer le comportement — dans le seul but de franchir la frontière de décision apprise par un modèle de classification, sans changer le comportement malveillant réel du fichier. Cette technique, qui exploite directement les faiblesses propres aux modèles de ML plutôt qu'une limite générale de l'analyse, est approfondie au chapitre 6.

7. Limites et bonnes pratiques

Un détecteur ML de malwares ne doit jamais être considéré comme une solution figée et définitive. Il doit être **régulièrement réentraîné** pour intégrer les nouvelles familles de malwares qui apparaissent en continu, et **évalué sur des échantillons réellement représentatifs** de la diversité rencontrée en conditions réelles — et pas uniquement sur des échantillons « faciles » qui surestimeraient artificiellement sa performance apparente. Enfin, il doit être **combiné avec des mécanismes complémentaires** — réputation de fichiers déjà connus, analyse comportementale poursuivie en production après le déploiement, renseignement sur les menaces (*threat intelligence*) partagé au sein de la communauté de sécurité — plutôt qu'utilisé comme unique et ultime ligne de défense.

À retenir

- L'analyse statique est rapide mais contournable par obfuscation ; l'analyse dynamique observe le comportement réel mais coûte plus cher et peut être détectée par le malware.
- Les modèles de classification supervisée sur caractéristiques extraites sont l'approche la plus répandue en pratique.
- Les techniques d'évasion (obfuscation, détection de sandbox, exemples adversariaux) rappellent qu'un détecteur ML n'est jamais une solution unique et définitive.

Chapitre 5 — NLP appliqué à la sécurité (phishing, analyse de logs)

IA appliquée à la sécurité informatique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le traitement automatique du langage naturel (NLP) en cybersécurité

Le NLP (*Natural Language Processing*) désigne l'ensemble des techniques permettant à un système informatique d'analyser, de comprendre ou de générer du langage humain. En cybersécurité, ces techniques permettent d'analyser automatiquement des contenus textuels très divers : e-mails, messages de messagerie instantanée, URL, journaux d'événements (logs) et rapports de renseignement sur les menaces (*threat intelligence*).

Parmi les nombreuses applications possibles du NLP en sécurité, deux se sont imposées comme particulièrement répandues dans l'industrie : la **détection de phishing**, qui exploite le contenu textuel des messages pour repérer des tentatives d'hameçonnage, et l'**analyse assistée de journaux volumineux**, qui permet de traiter des millions de lignes de logs comme du texte semi-structuré plutôt que de les examiner une par une. Ces deux applications sont détaillées respectivement dans les sections 3 et 5 de ce chapitre.

Un modèle de machine learning ne manipule jamais directement du texte brut : il travaille exclusivement sur des vecteurs de nombres. Il faut donc, avant tout entraînement, transformer chaque document textuel en une représentation numérique — une étape appelée **vectorisation** du texte. Plusieurs méthodes coexistent, avec des compromis très différents en termes de simplicité, de capacité à capturer le sens, et de coût de calcul.

Méthode	Principe	Caractéristique
Bag-of-words / TF-IDF	compte (pondéré) l'occurrence des mots, sans tenir compte de l'ordre	simple, efficace pour des tâches de classification classiques
N-grammes de caractères	séquences de n caractères consécutifs	robuste aux fautes d'orthographe volontaires (technique d'évasion courante en phishing)
Word embeddings (Word2Vec, GloVe)	représentation vectorielle dense capturant des relations sémantiques entre mots	capture le sens, plus coûteux à entraîner
Modèles de langage pré-entraînés (Transformers, type BERT)	représentation contextuelle, état de l'art actuel	performance élevée, coût de calcul plus important

L'approche **bag-of-words**, éventuellement pondérée par la mesure **TF-IDF** (*Term Frequency – Inverse Document Frequency*, qui donne plus de poids aux mots fréquents dans un document donné mais rares dans l'ensemble du corpus), représente un texte simplement par les mots qu'il contient et leur fréquence, en ignorant totalement leur ordre d'apparition. Cette simplicité en fait une méthode rapide à mettre en œuvre et souvent suffisamment efficace pour des tâches de classification classiques, malgré la perte évidente d'information sur la structure de la phrase.

Les **n-grammes de caractères** découpent le texte en séquences de n caractères consécutifs plutôt qu'en mots entiers. Cette granularité plus fine les rend particulièrement robustes face à une technique d'évasion très courante en phishing : l'introduction volontaire de fautes d'orthographe ou de caractères modifiés dans un mot sensible (par exemple pour tromper un filtre fondé sur une liste de mots-clés exacts), puisque la plupart des sous-séquences de caractères du mot original restent malgré tout présentes et détectables.

Les **word embeddings** (Word2Vec, GloVe) représentent chaque mot par un vecteur dense de nombres réels, appris de telle sorte que des mots au sens proche se retrouvent proches dans cet espace vectoriel. Cette représentation capture ainsi des relations sémantiques que le simple comptage de mots ne peut pas exprimer, au prix d'un entraînement plus coûteux et d'une nécessité de disposer d'un grand corpus de texte pour apprendre des représentations de qualité.

Les **modèles de langage pré-entraînés** de type Transformer (dont BERT est un représentant emblématique) représentent aujourd'hui l'état de l'art : contrairement aux word embeddings classiques, où chaque mot a une représentation fixe indépendante du contexte, ces modèles produisent une représentation **contextuelle**, qui varie selon les mots environnants (le mot « virus » n'est pas représenté de la même façon dans un contexte médical et dans un contexte informatique). Cette capacité se traduit généralement par de meilleures performances, au prix d'un coût de calcul

3. Détection de phishing par e-mail

La détection de phishing par e-mail combine typiquement plusieurs familles de caractéristiques, relevant à la fois du NLP et de l'analyse de métadonnées techniques :

- **contenu textuel du message** : présence d'un sentiment d'urgence artificiel, demande explicite d'une action (cliquer sur un lien, communiquer un mot de passe), fautes d'orthographe ou de grammaire, formulations incitant fortement au clic — autant de signaux linguistiques qu'un modèle NLP peut apprendre à reconnaître à partir d'exemples étiquetés ;
- **caractéristiques de l'expéditeur** : un nom de domaine récemment enregistré est statistiquement plus suspect qu'un domaine ancien et établi, de même qu'une usurpation du nom affiché de l'expéditeur (*display name spoofing*), où le nom visible dans le client de messagerie ne correspond pas à l'adresse réelle d'envoi ;
- **caractéristiques des URL contenues dans le message** : longueur inhabituelle de l'URL, présence de caractères trompeurs, ou similarité visuelle et lexicale forte avec un domaine légitime connu (technique dite du *typosquatting*, qui consiste à enregistrer un nom de domaine très proche d'un domaine légitime, avec une lettre substituée ou ajoutée) ;
- **en-têtes techniques du message** : les résultats des mécanismes d'authentification de messagerie (SPF, DKIM, DMARC) et d'éventuelles incohérences dans le chemin de routage du message peuvent trahir une usurpation d'identité de l'expéditeur, même lorsque le contenu textuel paraît anodin.

Une approche efficace combine généralement un modèle NLP appliqué au contenu du message avec des règles et caractéristiques dédiées à l'analyse des URL et des en-têtes techniques, plutôt que de s'appuyer sur un seul modèle qui traiterait l'ensemble du message comme un simple bloc de « texte brut » — les indices les plus fiables ne se trouvent pas tous dans le même type de signal.

4. Détection d'URL malveillantes

Une approche fréquente pour la détection d'URL malveillantes consiste en une classification supervisée fondée uniquement sur des **caractéristiques lexicales de l'URL elle-même** : sa longueur totale, le nombre de sous-domaines qu'elle contient, la présence de caractères spéciaux inhabituels, ou son entropie globale (une URL générée automatiquement par un outil de phishing tend à présenter une structure moins « naturelle » qu'une URL saisie par un humain). L'avantage majeur de cette approche est qu'elle ne nécessite pas nécessairement de visiter réellement la page cible pour se prononcer, ce qui la rend particulièrement adaptée à un **filtrage rapide en amont**, avant même qu'un utilisateur ne clique sur le lien — une analyse plus poussée (visite de la page dans un environnement contrôlé) pouvant ensuite être réservée aux cas jugés les plus ambigus par ce premier filtre.

5. Analyse de journaux (log analysis) assistée par NLP

Les journaux systèmes et applicatifs, bien que produits par des machines, peuvent être traités efficacement comme du **texte semi-structuré**, ouvrant la voie à plusieurs applications du NLP :

- **Regroupement de motifs (log clustering / log parsing)** : de nombreuses lignes de log, en apparence différentes, partagent en réalité un même « gabarit » (*template*) sous-jacent, avec seulement quelques valeurs variables (un identifiant, un horodatage, une adresse IP). Des outils spécialisés (comme **Drain**, un algorithme de parsing de logs largement utilisé) permettent de regrouper automatiquement des millions de lignes de logs en quelques centaines de motifs analysables, réduisant considérablement le volume à examiner manuellement par un analyste.
- **Détection d'anomalies séquentielles** : au-delà du contenu isolé de chaque ligne, l'ordre dans lequel les événements se succèdent porte lui-même une information précieuse. En modélisant la séquence normale d'événements attendue (à l'aide d'un modèle de séquence, comme évoqué au chapitre 4 pour les malwares), on peut détecter des déviations révélatrices d'un comportement d'attaque — par exemple, une séquence de commandes anormale exécutée juste après une connexion, potentiellement révélatrice d'une compromission en cours.
- **Résumé automatique** : face au volume considérable d'alertes générées par les outils de sécurité, des techniques de synthèse automatique — s'appuyant de plus en plus sur des modèles de langage génératifs — permettent de produire des résumés lisibles pour un analyste SOC. Cet usage impose cependant une vigilance particulière sur la fiabilité des sorties de ces modèles génératifs, un point approfondi au chapitre 6.

6. Limites spécifiques au NLP en sécurité

Le NLP appliqué à la sécurité présente des limites qui lui sont propres, distinctes de celles déjà évoquées pour la détection de malwares ou d'intrusions réseau.

Les attaquants **adaptent activement leur texte** pour échapper à la détection : fautes d'orthographe volontaires, remplacement de lettres par des caractères visuellement similaires mais techniquement différents (homoglyphes, par exemple un « o » latin remplacé par un caractère cyrillique de forme identique), ou dissimulation de texte à l'intérieur d'images pour échapper à une analyse purement textuelle. Ce phénomène constitue un jeu du chat et de la souris directement comparable à celui déjà décrit pour les techniques d'évasion des détecteurs de malwares au chapitre 4.

Un modèle NLP entraîné sur une langue ou un contexte culturel donné **généralise mal** à d'autres langues ou à d'autres contextes culturels : les indicateurs textuels pertinents pour repérer un e-mail de phishing rédigé dans une langue peuvent ne pas se transposer directement à une autre langue, ce qui impose une vigilance particulière lors du déploiement d'un tel modèle dans un contexte multilingue.

Enfin, l'essor des **modèles de langage génératifs**, utilisés de façon offensive par des attaquants pour rédiger des messages de phishing très convaincants, sans fautes d'orthographe et potentiellement en plusieurs langues, réduit sensiblement l'efficacité des indicateurs textuels classiques (fautes d'orthographe, formulations maladroites) longtemps utilisés pour repérer le phishing. Cette évolution pousse progressivement les défenses à s'appuyer davantage sur des signaux techniques complémentaires (URL, en-têtes, authentification du domaine, comme vu en section 3) plutôt que sur le seul contenu textuel du message.

À retenir

- Le NLP transforme du texte en représentation numérique exploitable par un modèle (TF-IDF, embeddings, Transformers), avec un compromis performance/coût de calcul.
- La détection de phishing combine efficacement analyse du contenu textuel et caractéristiques techniques (URL, en-têtes, authentification du domaine).
- L'essor des modèles de langage génératifs facilite la production de contenus de phishing convaincants, ce qui affaiblit les indicateurs textuels traditionnels et pousse vers des défenses combinant plusieurs signaux.

Chapitre 6 — Sécurité de l'IA : attaques adversariales et IA offensive

IA appliquée à la sécurité informatique

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. L'IA comme nouvelle surface d'attaque

À mesure que des systèmes d'IA sont déployés dans des fonctions de plus en plus sensibles — détection de fraude bancaire, contrôle d'accès biométrique, filtrage automatique de contenu, systèmes autonomes — ils cessent d'être uniquement des outils de défense pour devenir eux-mêmes des **cibles** à part entière. Un système d'IA est, après tout, un logiciel qui reçoit des entrées et produit des sorties ; il hérite à ce titre de nouvelles catégories de vulnérabilités, distinctes des vulnérabilités logicielles classiques (débordement de mémoire, injection SQL, etc.) déjà étudiées ailleurs dans le programme.

Cette classe émergente de menaces est aujourd'hui recensée et structurée par des référentiels dédiés, au même titre que l'OWASP Top 10 structure les vulnérabilités web classiques : le **OWASP Machine Learning Security Top 10** répertorie les dix catégories de risques les plus critiques spécifiques au ML, tandis que le cadre **MITRE ATLAS** (*Adversarial Threat Landscape for Artificial-Intelligence Systems*) documente, sur le modèle de MITRE ATT&CK, les tactiques et techniques employées par des attaquants ciblant spécifiquement des systèmes d'IA. Ce chapitre reprend les grandes catégories de menaces qu'ils recensent.

Le principe d'une attaque adversariale est de modifier légèrement une entrée soumise à un modèle déjà entraîné — une modification souvent imperceptible ou peu remarquable pour un observateur humain — dans le seul but de tromper le modèle et de lui faire produire une prédiction erronée, **sans modifier le comportement réel de l'objet analysé**. C'est cette dissociation entre l'apparence perçue par un humain et la prédiction produite par le modèle qui rend ces attaques particulièrement préoccupantes.

- **Sur des images** : un exemple classique, largement documenté dans la littérature académique sur l'apprentissage adversarial, consiste à ajouter à une image un bruit numérique calculé avec précision, invisible ou quasiment invisible à l'œil nu, qui suffit à faire classer par un modèle de vision une image de panneau stop comme un panneau de limitation de vitesse — un cas d'école qui illustre bien les conséquences potentielles pour des systèmes de vision embarqués dans des contextes critiques.
- **Sur des malwares** : comme évoqué au chapitre 4, ajouter des octets inoffensifs à un binaire malveillant, ou en réorganiser légèrement certaines sections, peut suffire à franchir la frontière de décision d'un classifieur de malwares, sans changer en rien le comportement malveillant réel du fichier une fois exécuté.
- **Sur du texte (phishing, spam)** : de la même façon, introduire des variations orthographiques ciblées, des caractères invisibles (par exemple des caractères Unicode sans effet visuel), ou remplacer certains mots par des synonymes suffit souvent à échapper à un filtre NLP, tout en laissant le message parfaitement compréhensible — et donc tout aussi efficace — pour un destinataire humain.

Modèles d'attaque

La faisabilité pratique d'une attaque adversariale dépend fortement de ce que l'attaquant sait du modèle ciblé — une distinction structurante dans la littérature sur le sujet :

Modèle	Connaissance de l'attaquant
Boîte blanche (white-box)	accès complet au modèle (architecture, paramètres) — permet de calculer des perturbations optimales
Boîte noire (black-box)	accès uniquement aux prédictions du modèle (requêtes successives) — attaques par substitution (entraîner un modèle de substitution pour approximer les frontières de décision)

En **boîte blanche**, l'attaquant dispose d'un accès complet à l'architecture et aux paramètres internes du modèle ciblé, ce qui lui permet de calculer mathématiquement, par des méthodes d'optimisation, la perturbation minimale nécessaire pour franchir sa frontière de décision — le scénario le plus favorable à l'attaquant, mais aussi le moins réaliste dans un contexte où le modèle n'est en principe pas exposé publiquement.

En **boîte noire**, situation nettement plus représentative des conditions réelles d'exploitation d'une API de modèle exposée à des tiers, l'attaquant n'observe que les prédictions produites en réponse à ses propres requêtes, sans jamais accéder aux paramètres internes. Une stratégie courante consiste alors à mener des **attaques par substitution** : entraîner son propre modèle de substitution à partir des couples entrée/sortie observés,

3. Empoisonnement de données (data poisoning)

Contrairement aux attaques adversariales, qui ciblent un modèle déjà entraîné au moment de l'inférence, l'**empoisonnement de données** vise directement la **phase d'entraînement** elle-même. Un attaquant capable d'injecter des données malveillantes dans le jeu d'entraînement d'un modèle — un scénario réaliste notamment pour tout système qui se ré-entraîne en continu sur des données fournies ou influencées par des utilisateurs — peut poursuivre deux objectifs distincts :

- **dégrader la performance générale** du modèle, en y introduisant suffisamment de données bruitées ou mal étiquetées pour altérer sa capacité de généralisation sur l'ensemble de sa mission ;
- ou, de façon plus ciblée et plus insidieuse, y insérer une **porte dérobée (backdoor)** : le modèle continue alors à se comporter parfaitement normalement sur la quasi-totalité des entrées, mais produit une prédiction spécifiquement choisie par l'attaquant lorsqu'un déclencheur particulier — connu de lui seul — est présent dans l'entrée, un déclencheur qui peut être aussi discret qu'un motif visuel précis ou une combinaison de caractéristiques rares.

Ce second scénario est particulièrement redoutable précisément parce qu'il est difficile à détecter par une simple évaluation globale du modèle : un modèle « backdooré » peut afficher une excellente performance sur l'ensemble des métriques standards (précision, rappel, F1-score) tout en restant compromis pour le petit sous-ensemble d'entrées porteuses du déclencheur choisi par l'attaquant.

4. Extraction et inversion de modèle

Au-delà de la tromperie du modèle (evasion) et de la corruption de son entraînement (poisoning), une troisième famille de menaces concerne directement la **confidentialité** du modèle et des données ayant servi à l'entraîner.

- **Extraction de modèle (model extraction)** : en interrogeant massivement l'API d'un modèle propriétaire et en observant systématiquement ses réponses, un attaquant peut progressivement reconstruire une approximation fonctionnelle du modèle original — une opération qui permet à la fois de contourner les coûts de développement engagés par le propriétaire légitime (vol de propriété intellectuelle) et de préparer, comme évoqué en section 2, une attaque adversariale en boîte blanche sur cette copie reconstituée, transférable ensuite au modèle original.
- **Inversion de modèle / attaque par appartenance (membership inference)** : cette famille d'attaques cherche à déterminer si une donnée spécifique donnée a été utilisée dans le jeu d'entraînement d'un modèle (attaque par appartenance), ou, plus radicalement, à reconstruire partiellement des données d'entraînement sensibles à partir du seul comportement du modèle entraîné. Ces attaques constituent un enjeu direct de confidentialité, en particulier au regard de réglementations comme le RGPD, dès lors qu'un modèle a été entraîné sur des données à caractère personnel : savoir qu'une personne donnée figurait dans le jeu d'entraînement peut en soi révéler une information sensible sur elle (par exemple son appartenance à une cohorte médicale spécifique).

5. IA offensive : usage par les attaquants

Au-delà des attaques qui ciblent directement des systèmes d'IA, les attaquants utilisent également l'IA comme un **outil** au service de leurs propres opérations offensives, ce qui modifie sensiblement l'équilibre traditionnel entre attaquants et défenseurs :

- **génération automatisée de contenus de phishing** personnalisés et rédigés sans fautes, produits à grande échelle et adaptés au contexte de chaque cible, réduisant l'efficacité des indicateurs textuels classiques déjà évoqués au chapitre 5 ;
- **assistance à l'écriture de code malveillant** ou à l'automatisation de certaines tâches de reconnaissance préalables à une attaque, réduisant le niveau de compétence technique nécessaire pour mener certaines opérations ;
- **génération de deepfakes** (voix, image, vidéo synthétiques) utilisés dans des fraudes ciblées — par exemple l'usurpation de la voix d'un dirigeant d'entreprise pour tenter d'autoriser frauduleusement un virement bancaire, une technique d'ingénierie sociale qui exploite la confiance naturellement accordée à une voix reconnue ;
- **automatisation partielle** de certaines phases de reconnaissance ou de la rédaction de rapports d'attaque par les attaquants eux-mêmes, accélérant le rythme global de leurs opérations.

6. Contre-mesures et bonnes pratiques

Face à ces menaces spécifiques, plusieurs familles de contre-mesures ont émergé, résumées dans le tableau suivant. Aucune n'est infaillible prise isolément : une défense robuste combine généralement plusieurs de ces mesures.

Menace	Contre-mesure
Attaques adversariales	entraînement adversarial (inclure des exemples adversariaux dans l'entraînement), détection de perturbations anormales, limitation du taux de requêtes à une API de modèle
Empoisonnement de données	validation et traçabilité des sources de données d'entraînement, détection d'anomalies dans les données avant réentraînement
Extraction/inversion de modèle	limitation et surveillance des requêtes API, techniques de confidentialité différentielle
IA offensive (phishing, deepfakes)	sensibilisation renforcée, vérification hors bande des demandes sensibles (procédures « callback » avant tout virement), défenses multi-signaux ne reposant pas uniquement sur des indices textuels

L'**entraînement adversarial** consiste à intégrer volontairement des exemples adversariaux dans le jeu d'entraînement du modèle, afin qu'il apprenne à rester robuste face à ce type de perturbation — une forme de vaccination du modèle contre les attaques qu'il est susceptible de rencontrer en production. La **validation et la traçabilité des sources de données d'entraînement** limitent la surface d'exposition à l'empoisonnement, en s'assurant qu'on peut identifier et écarter une source de données compromise. La **confidentialité différentielle** (*differential privacy*), une technique qui introduit un bruit statistique contrôlé lors de l'entraînement, réduit la capacité d'un attaquant à inférer des informations précises sur des données d'entraînement individuelles. Enfin, face à l'IA offensive, les contre-mesures les plus efficaces restent souvent organisationnelles autant que techniques : une procédure de vérification hors bande (un appel téléphonique de confirmation avant tout virement sensible, par exemple) neutralise l'essentiel de la menace des deepfakes vocaux, quelle que soit la sophistication technique de l'attaque.

À retenir

- Les systèmes d'IA sont vulnérables à des classes d'attaques spécifiques (adversariales, empoisonnement, extraction) distinctes des vulnérabilités logicielles classiques.
- Les modèles de connaissance de l'attaquant (boîte blanche/noire) déterminent la faisabilité pratique de ces attaques.
- L'IA offensive (contenus de phishing, deepfakes) accroît la sophistication des attaques d'ingénierie sociale et impose des contre-mesures organisationnelles autant que techniques.